

Agent 工具调用机制学习笔记

围绕 todo_write、messages/history、tool_use、解包、*args/**kwargs、左值右值的系统梳理

核心主线

这次对话的主线是：LLM 通过 tool_use 生成结构化工具参数，Python Agent 通过 dispatch map 找到 handler，再用解包语法把 input dict 转成函数参数，最后把 tool_result 追加回 history。

主题	一句话总结
todo_write	一个普通工具，用来更新 CURRENT_TODOS 并在终端显示进度。
messages/history	对话上下文列表，保存 user/assistant/tool_result 的顺序记录。
tool_use.input	LLM 按 input_schema 生成的工具参数字典。
解包	把返回值或容器结构拆开，交给变量或函数参数。
左值/右值	Python 中更准确叫 assignment target 和 expression。

1. 本次对话的核心结论

重点结论

不要把 `todo_write` 的核心逻辑写进 `PreToolUse`。`todo_write` 应该是普通工具；`PreToolUse` 是工具执行前的拦截/检查 hook。

你一开始问的是 `todo_write` 工具的执行机制。最终可以归纳成一句话：

`todo_write` 是 Agent 用来显式维护任务进度的普通工具；它通过 `tool_use` 被 LLM 调用，通过 `TOOL_HANDLERS` 被 Python 分发执行，通过 `tool_result` 把执行结果放回 `messages/history`。

这里要分清三个层次：

层次	代表数据	职责
LLM 上下文层	<code>history/messages</code>	保存对话历史，让模型知道前面发生了什么。
工具参数层	<code>block.input</code>	保存某一次工具调用的参数。
Python 状态层	<code>CURRENT_TODOS</code>	保存当前进程内存里的 <code>todo</code> 状态，并用于终端展示。

思维导图 1：todo_write 在 Agent 系统中的位置

todo_write = 普通工具 + 状态同步入口	
LLM 根据用户任务决定是否调用 <code>todo_write</code> 。	<code>input_schema</code> 约束 <code>input</code> 结构，告诉 LLM 应该生成 <code>todos</code> 参数。
<code>TOOL_HANDLERS</code> 把工具名 <code>todo_write</code> 映射到 <code>run_todo_write</code> 。	<code>CURRENT_TODOS</code> 当前 Python 进程内存里的任务列表。
<code>print()</code> 给终端用户看进度面板。	<code>return</code> 生成 <code>tool_result</code> ，再追加回 <code>history</code> 。

这就解释了为什么 `todo_write` 不是 `PreToolUse` 本身：`PreToolUse` 是“所有工具执行前的安检门”，`todo_write` 是“被安检的一个具体工具”。

2. todo_write 的作用与数据结构

原始版本的 `todo_write` 逻辑大概是：接收一个 `todos` 列表，保存到全局变量 `CURRENT_TODOS`，然后打印任务进度。

```
CURRENT_TODOS: list[dict] = []

def run_todo_write(todos: list) -> str:
    global CURRENT_TODOS
    CURRENT_TODOS = todos

    lines = ["
## Current Tasks"]
    for t in CURRENT_TODOS:
        icon = {"pending": " ", "in_progress": ">", "completed": "√"}[t["status"]]
        lines.append(f" [{icon}] {t['content']}")

    print("
".join(lines))
    return f"Updated {len(CURRENT_TODOS)} tasks"
```

这个函数有两个输出通道：

代码	输出给谁	作用
print("\n".join(lines))	终端用户	显示任务看板。
return "Updated N tasks"	Agent/LLM	作为 tool_result 放回 messages。

重点

CURRENT_TODOS 只存在于当前 Python 进程内存中。它不会自动进入 LLM 上下文；LLM 真正能看到的是 messages/history 里的 tool_use 和 tool_result。

每个 todo 是一个 dict，整个 todos 是 list[dict]：

```
todos = [
    {"content": "Read README.md", "status": "completed"},
    {"content": "Fix encoding issue", "status": "in_progress"},
    {"content": "Run tests", "status": "pending"},
]
```

其中 status 只能是三种状态：

status	含义	显示
pending	等待执行	[]
in_progress	正在执行	[>]
completed	已经完成	[√]

3. 工具定义、schema 与 dispatch map

工具定义分两部分：一部分给 LLM 看，另一部分给 Python 程序执行。

结构	面向谁	作用
TOOLS	LLM	说明有哪些工具、每个工具需要什么参数。
input_schema	LLM	约束工具 input 的 JSON 结构。
TOOL_HANDLERS	Python	把工具名映射到真正的 Python 函数。

例如 `todo_write` 的 `input_schema` 会告诉模型：调用这个工具时，`input` 里要有 `todos` 字段。

```
TOOLS = [
    {
        "name": "todo_write",
        "description": "Create and manage a task list",
        "input_schema": {
            "type": "object",
            "properties": {
                "todos": {
                    "type": "array",
                    "items": {
                        "type": "object",
                        "properties": {
                            "content": {"type": "string"},
                            "status": {
                                "type": "string",
                                "enum": ["pending", "in_progress", "completed"]
                            }
                        },
                    },
                },
                "required": ["content", "status"]
            },
        },
        "required": ["todos"]
    },
]

TOOL_HANDLERS["todo_write"] = run_todo_write
```

工程重点

`input_schema` 里的字段名、LLM 生成的 `block.input` 的 key、Python handler 的参数名必须一致。例如 `todos` -> `block.input["todos"]` -> `run_todo_write(todos)`。

思维导图 2：schema 到函数参数的映射

input_schema -> block.input -> handler(**input)	
TOOLS 把工具能力暴露给 LLM。	input_schema 规定 input 必须是 object/dict。
LLM 输出 生成 tool_use，其中包含 name 和 input。	block.name 决定调用哪个工具。
block.input 决定传什么参数。	**block.input 把 dict 展开成关键字参数。

4. Agent 调用链：从用户输入到 todo 更新

你问过：用户输入请求后，大模型第一次返回 content，sync_todo_list 被调用更新 todo，然后再喂给大模型，对吗？修正后的流程如下：

```

用户输入
|
v
history.append({"role": "user", "content": user_input})
|
v
client.messages.create(messages=history, tools=TOOLS)
|
v
LLM 返回 assistant message
|
+-- 如果 content 中有 tool_use: todo_write
    |
    v
    agent loop 解析 block.name / block.input
    |
    v
    handler = TOOL_HANDLERS[block.name]
    |
    v
    result = handler(**block.input)
    |
    v
    run_todo_write(todos=[...])
    |
    +-- CURRENT_TODOs = todos
    +-- print 当前任务进度
    +-- return "Updated N tasks"
    |
    v
    history.append({"role": "user", "content": [tool_result]})
    |
    v
    再次调用 LLM(messages=history, tools=TOOLS)
```

一个典型的 tool_use block 如下：

```
{
  "type": "tool_use",
  "id": "toolu_001",
  "name": "todo_write",
  "input": {
    "todos": [
      {"content": "Find Python files", "status": "in_progress"},
      {"content": "Summarize result", "status": "pending"}
    ]
  }
}
```

这时：

表达式	值	意义
block.name	"todo_write"	工具名。
block.input	{"todos": [...]}	工具参数字典。
TOOL_HANDLERS[block.name]	run_todo_write	真正要调用的 Python 函数。
handler(**block.input)	run_todo_write(todos=[...])	把 input dict 展开后调用函数。

重点修正

todo 不是靠普通 content 自动同步的，而是模型返回 tool_use 后，agent loop 解析并执行对应 handler。执行结果再作为 tool_result 放回 history。

5. 为什么大量使用 list + dict 结构？

你的理解“这些数据统一成列表，方便后续匹配和存取”基本正确，但更准确是：统一成 JSON 风格的 list + dict。

更准确的说法

多个同类记录用 list；每条记录的字段用 dict。也就是：外层 list 保持顺序和遍历，内层 dict 支持按字段匹配和取值。

数据	典型类型	为什么这样设计
history/messages	list[dict]	多条有顺序的对话消息。
response.content	list[dict]	一次模型回复中可能包含多个 text/tool_use block。
tool_use.input	dict	一组函数参数，适合用 ** 展开。
input["todos"]	list[dict]	多个任务，每个任务有 content/status 字段。
CURRENT_TODOS	list[dict]	当前任务列表，方便遍历显示。

思维导图 3：JSON 风格结构的设计规律

list + dict = 顺序 + 字段	
list 适合保存多条记录，例如 messages、todos、content blocks。	dict 适合保存一条记录的多个字段，例如 role/content/name/input。
schema 规定 dict 里应该有哪些字段，字段是什么类型。	遍历 for message in history / for t in todos。
匹配 if block["type"] == "tool_use"。	取值 block["input"]["todos"] / t["status"]。

这个设计和 JSON/API 非常契合，因为 list、dict、str、int、bool、None 都很容易序列化。LLM 工具调用本质上也是在生产 JSON 风格的结构化数据。

6. _normalize_todos：参数规范化与校验

你给出的改进版增加了 _normalize_todos，用来处理 LLM 可能传错格式的问题。

```
def _normalize_todos(todos):
    if isinstance(todos, str):
        try:
            todos = json.loads(todos)
        except json.JSONDecodeError:
            try:
                todos = ast.literal_eval(todos)
            except (SyntaxError, ValueError):
                return None, "Error: todos must be a list or JSON array string"

    if not isinstance(todos, list):
        return None, "Error: todos must be a list"

    for i, t in enumerate(todos):
        if not isinstance(t, dict):
            return None, f"Error: todos[{i}] must be an object"
        if "content" not in t or "status" not in t:
            return None, f"Error: todos[{i}] missing 'content' or 'status'"
        if t["status"] not in ("pending", "in_progress", "completed"):
            return None, f"Error: todos[{i}] has invalid status '{t['status']}'"

    return todos, None
```

它做了四类工作：

步骤	代码机制	目的
字符串解析	json.loads / ast.literal_eval	把字符串形式的列表变成真正的 Python list。
整体类型检查	isinstance(todos, list)	确保 todos 是任务列表。
元素类型检查	isinstance(t, dict)	确保每个任务是对象/字典。
字段与状态检查	content/status + enum	确保每个任务字段完整且状态合法。

安全点

ast.literal_eval 比 eval 安全得多，因为它只解析 Python 字面量，不执行任意代码。

7. 解包知识点：tuple unpacking、*args、**kwargs

这次重点涉及三种“拆开”的语法：赋值解包、调用时的 * 展开、调用时的 ** 展开。

7.1 赋值解包：todos, error = ...

```
todos, error = _normalize_todos(todos)
```

右边函数返回两个值，本质是一个 tuple：

```
return todos, None
# 等价于
return (todos, None)
```

左边的两个变量按位置接收：

```

_normalize_todos(todos)
|
v
(normalized_todos, error_msg)
|
+----> todos
|
+----> error
```

7.2 调用时的 ** 字典展开

```
handler(**block.input)
```

如果 block.input 是：

```
block.input = {
    "todos": [
        {"content": "Read file", "status": "pending"}
    ]
}
```


那么：

```
handler(**block.input)
# 等价于
handler(todos=[{"content": "Read file", "status": "pending"}])
```

这就是 Agent 工具系统喜欢使用 input dict 的原因：不同工具的参数不同，但都可以统一通过 dict 展开成关键字参数。

7.3 调用时的 * 列表/元组展开

```
def add(a, b):
    return a + b

nums = [1, 2]
add(*nums)
# 等价于 add(1, 2)
```

7.4 函数定义时的 *args / **kwargs

注意：函数调用时是“拆开”，函数定义时是“收集”。

场景	写法	含义
调用函数	func(*my_list)	把 list/tuple 展开成多个位置参数。
调用函数	func(**my_dict)	把 dict 展开成多个关键字参数。
定义函数	def func(*args)	把多个位置参数收集成 tuple。
定义函数	def func(**kwargs)	把多个关键字参数收集成 dict。

思维导图 4：Python 解包语法分类

解包 = 把结构拆开，或把参数收集起来	
赋值解包 a, b = result；右边产生多个值，左边多个目标接收。	调用 * func(*list)；list/tuple -> 位置参数。
调用 ** func(**dict)；dict -> 关键字参数。	定义 *args 收集任意位置参数为 tuple。
定义 **kwargs 收集任意关键字参数为 dict。	Agent 用法 handler(**block.input) 统一执行不同工具。

8. 解包和左值/右值有什么关系？

有关系，但要分清：在 Python 中更常说 assignment target 和 expression，而不是 C/C++ 语境下严格的左值/右值。

传统说法	Python 更准确说法	例子
左值	assignment target / 赋值目标	todos, error
右值	expression / 表达式	_normalize_todos(todos)

赋值解包可以用左值/右值理解：

```
todos, error = _normalize_todos(todos)
```

右边 expression 先执行：
_normalize_todos(todos)
-> 返回 (normalized_todos, error_msg)

左边 assignment target 接收：
todos <- normalized_todos
error <- error_msg

但是 handler(**block.input) 不是左值/右值赋值，它是函数调用时的参数展开。

```
result = handler(**block.input)
```

这里可以拆成两层理解：

部分	含义
handler(**block.input)	右边表达式：调用函数并产生 result。
**block.input	调用参数展开：把 dict 拆成关键字参数。
result	赋值目标：接收函数返回值。

一句话记忆

todos, error = ... 是赋值解包，和左值/右值关系很强；handler(**block.input) 是函数调用参数展开，重点不是赋值，而是把 dict 转换成关键字参数。

9. 常见易错点与工程建议

易错点	错误理解	正确理解
todo_write 与 PreToolUse	把 todo_write 放进 PreToolUse 里。	todo_write 是普通工具；PreToolUse 是工具执行前的 hook。
CURRENT_TODOS 与 history	CURRENT_TODOS 自动被 LLM 看到。	LLM 只能看到 messages/history 中的内容。
统一为 list	所有数据都应该是 list。	更准确是 JSON 风格：多条记录用 list，一条记录的字段用 dict。
block.input	input 是用户直接输入的。	input 是 LLM 根据 input_schema 生成的工具参数 dict。
**block.input	随便什么 dict 都能传。	dict 的 key 必须匹配 handler 的参数名。
解包与左值右值	所有解包都是左值右值。	赋值解包相关；函数调用展开不完全属于这个视角。

推荐的工具执行函数结构如下：

```
def execute_tool(name: str, args: dict) -> str:
    run_hook("PreToolUse", name, args) # 只做检查/拦截/日志

    handler = TOOL_HANDLERS[name]
    result = handler(**args) # 统一把 input dict 展开成参数

    run_hook("PostToolUse", name, args, result)
    return result
```

推荐的 todo_write 分层结构如下：

```
def sync_todo_list(todos):
    global CURRENT_TODOS
    CURRENT_TODOS = todos

def print_todos(todos):
    lines = [
        ## Current Tasks"]
        for t in todos:
            icon = {"pending": " ", "in_progress": ">", "completed": "√"}[t["status"]]
            lines.append(f" [{icon}] {t['content']}")
        print("".join(lines))

def run_todo_write(todos: list | str) -> str:
    todos, error = _normalize_todos(todos)
    if error:
        return error

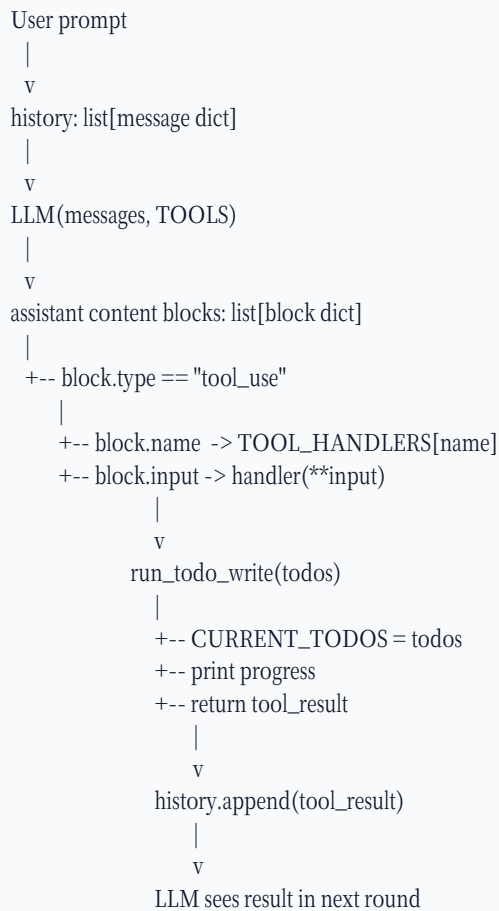
    sync_todo_list(todos)
    print_todos(todos)
    return f"Updated {len(todos)} tasks"
```

10. 最终速记版

最重要的 6 句话

1. `todo_write` 是普通工具，不是 `PreToolUse`。
2. LLM 通过 `tool_use` 调用工具；Python 通过 `TOOL_HANDLERS` 执行工具。
3. `block.name` 决定工具名；`block.input` 决定工具参数。
4. `handler(**block.input)` 把 `input dict` 展开成关键字参数。
5. `messages/history` 是 LLM 上下文；`CURRENT_TODOS` 是 Python 内存状态。
6. `list` 用来保存多条记录，`dict` 用来保存一条记录的字段。

最终总图：



到这里，你可以把 Agent 工具调用系统理解成一个“结构化消息循环”：LLM 负责产生结构化意图，Python 负责按工具表执行，`history` 负责记录上下文，`CURRENT_TODOS` 负责保存运行时状态，而解包语法负责把 JSON 风格的 `input` 平滑转换成 Python 函数参数。